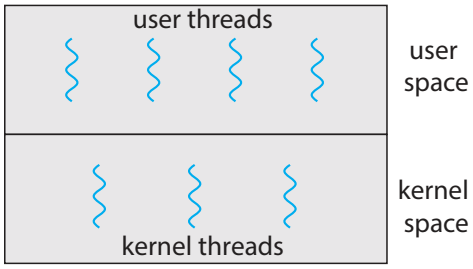
۴-۳ مدل‌های چندنخی

تاکنون در مباحثی که داشتیم، حالت کلی نخ‌ها مطرح بود. با این وجود، پشتیبانی نخ‌ها یا در سطح کاربر فراهم می‌شود که **نخ‌های کاربر** و یا در هسته مهیا می‌شود که **نخ‌های هسته**[1] نامیده می‌شوند. نخ‌های کاربری در بالای هسته پشتیبانی می‌شوند و بدون پشتیبانی هسته اداره می‌شوند، در حالی که نخ‌های هسته به طور مستقیم توسط سیستم‌عامل، پشتیبانی و مدیریت می‌شوند. تقریباً، تمامی سیستم‌عامل‌های کنونی—اعم از ویندوز، لینوکس، و macOS—نخ‌های هسته را پشتیبانی می‌کنند.

در نهایت، بایستی رابطه‌ای میان نخ‌های کاربر و نخ‌های هسته وجود داشته باشد، آن‌گونه که در شکل ۴-۶ نشان داده می‌شود. در این بخش، به سه روش رایج برپایی چنین رابطه‌ای می‌پردازیم که عبارتند از: مدل چند-به-یک، مدل یک-به-یک و مدل چند-به-چند.

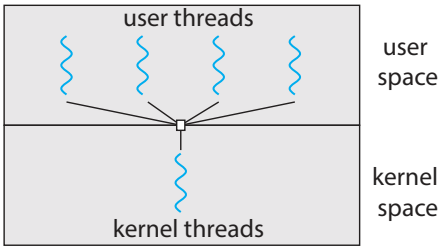


شکل ۴-۶ نخ‌های کاربر و هسته.

۴-۳-۱ مدل چند-به-یک

مدل چند-به-یک (شکل ۴-۷)، نخ‌های زیادی از سطح کاربر را به یک نخ هسته نگاشت می‌دهد. مدیریت نخ این مدل، توسط کتابخانه‌ی نخ فضای کاربر انجام می‌شود، لذا کارآمد است (کتابخانه‌های نخ را در بخش ۴-۴ بحث می‌کنیم). با این همه، اگر نخی یک فراخوانی سیستمی مسدودشونده انجام بدهد، کل پردازه مسدود خواهد شد. همچنین، چون در هر لحظه تنها یک نخ می‌تواند به هسته دسترسی داشته باشد، نخ‌های متعدد نمی‌توانند روی سیستم‌های چندهسته‌ای اجرای موازی داشته باشند. **نخ‌های سبز**—کتابخانه‌ی نخی که برای سیستم‌های سولاریس فراهم است و در نسخه‌های اولیه‌ی جاوا استفاده می‌شد—از مدل چند-به-یک استفاده کرد. با این وجود، سیستم‌های بسیار کمی، به دلیل عدم توانایی آن در بهره‌مندی از مزیت هسته‌های محاسباتی متعدد، به استفاده از این مدل ادامه دادند که اکنون در اکثر سیستم‌های کامپیوتری استاندارد شده است.

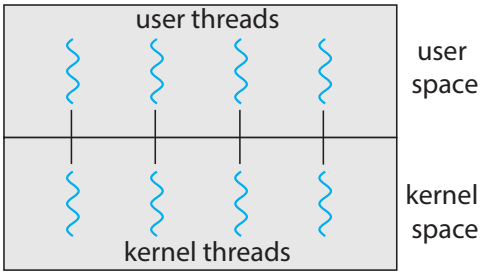
1. Kernel threads



شکل ۴-۷ مدل چند-به-یک.

۴-۳-۲ مدل یک-به-یک

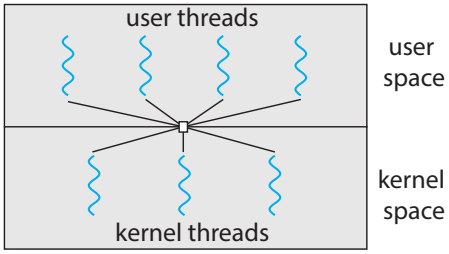
مدل یک-به-یک (شکل ۴-۸)، هر یک از نخ‌های کاربر را به یک نخ هسته نگاشت می‌دهد. با این مدل، زمانی که نخی فراخوانی سیستمی مسدودشونده انجام بدهد، نخ دیگری می‌تواند اجرا داشته باشد، لذا همروندی آن از مدل چند-به-یک بیشتر خواهد بود. همین‌طور، اجرای موازی نخ‌های متعدد را در چندپردازنده‌ها فراهم می‌کند. تنها عیب این مدل آن است که برای ایجاد یک نخ کاربر لازم است نخ متناظر هسته نیز ایجاد شود، و تعداد زیادی نخ‌های هسته ممکن است کارایی یک سیستم را بسیار پایین بیاورد. لینوکس در کنار خانواده‌ی سیستم‌عامل‌های ویندوز، مدل یک-به-یک را پیاده‌سازی می‌کنند.



شکل ۴-۸ مدل یک-به-یک.

۴-۳-۳ مدل چند-به-چند

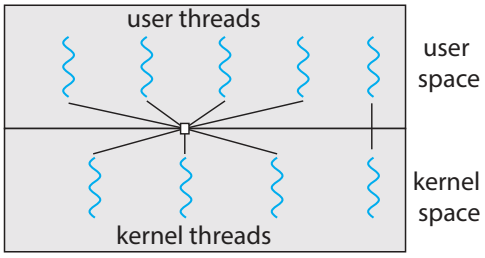
مدل چند-به-چند (شکل ۴-۹)، نخ‌های متعددی را از سطح کاربر به تعداد کمتر یا مساوی از نخ‌های هسته مالتی‌پلکس می‌کند. تعداد نخ‌های هسته می‌توانند خاص یک برنامه‌ی کاربردی مشخص و یا ماشین به‌خصوصی باشند (یک برنامه‌ی کاربردی در یک سیستم با هشت هسته‌ی پردازشی نسبت به یک سیستم با چهار هسته ممکن است نخ‌های هسته‌ی بیشتری در اختیار بگیرد).



شکل ۴-۹ مدل چند-به-چند.

بیایید تأثیر این طراحی را روی همروندی بررسی کنیم. در حالی که مدل چند-به-یک به برنامه‌نویس اجازه می‌دهد تا به هر تعداد نخ کاربری که بخواهد ایجاد کند، به موازی‌سازی منجر نمی‌شود، چون هسته در هر لحظه می‌تواند فقط یک نخ هسته را زمانبندی کند. مدل یک-به-یک همروندی بیشتری را عاید می‌سازد، اما برنامه‌نویس بایستی دقت داشته باشد که در یک برنامه‌ی کاربردی، نخ زیادی ایجاد نکند. (در حقیقت، در بعضی سیستم‌ها، ممکن است در تعداد نخ‌هایی که می‌تواند ایجاد کند محدود شود). مدل چند-به-چند هیچ یک از این مشکلات را ندارد: برنامه‌نویسان می‌توانند به هر تعداد که لازم باشد نخ کاربری ایجاد کنند و نخ‌های متناظر هسته در یک چندپردازنده می‌توانند اجرای موازی داشته باشند. همچنین، زمانی که نخی یک فراخوانی سیستمی مسدودشونده انجام می‌دهد، هسته می‌تواند نخ دیگری را برای اجرا زمانبندی کند.

یک گونه از مدل چند-به-چند وجود دارد که در حالی که نخ‌های سطح کاربر متعددی را به تعداد کمتر یا مساوی از نخ‌های هسته مالتی‌پلکس می‌کند، ولی این امکان را فراهم می‌کند که یک نخ سطح کاربر به یک نخ هسته پابند شود. گاهی از این گونه به نام *مدل دوسطحی* یاد می‌شود (شکل ۴-۱۰).



شکل ۴-۱۰ مدل دوسطحی.

اگرچه به‌ظاهر در بین مدل‌هایی که بحث شدند، مدل چند-به-چند انعطاف بیشتری دارد، پیاده‌سازی آن در عمل مشکل است. به‌علاوه، با ازدیاد تعداد هسته‌های پردازشی که روی اکثر سیستم‌ها وجود دارند، اهمیت محدودساختن تعداد نخ‌های هسته کمتر شده است. در نتیجه، اکثر سیستم‌عامل‌ها هم‌اکنون از مدل یک-به-یک استفاده می‌کنند. با این وجود، همان‌گونه که در بخش ۴-۵ خواهیم دید، برخی کتابخانه‌های همروندی کنونی، توسعه‌دهندگان را وادار می‌کنند تا وظایفی را شناسایی کنند که در ادامه با استفاده از مدل چند-به-چند به نخ‌ها نگاشته شوند.

۴-۴ کتابخانه‌های نخ

کتابخانه‌ی نخ، یک API برای برنامه‌نویس جهت ایجاد و مدیریت نخ‌ها فراهم می‌کند. دو روش اصلی برای پیاده‌سازی کتابخانه‌ی نخ وجود دارند. رویکرد اول آن است که کتابخانه به طور کامل در فضای کاربر بدون هیچ‌گونه پشتیبانی از جانب هسته فراهم شود. تمامی کد و ساختمان‌داده‌های کتابخانه در فضای کاربر قرار دارند. این بدان معنی است که پیامد احضار تابعی از کتابخانه، یک فراخوانی تابع محلی در فضای کاربر است و فراخوانی سیستمی نخواهد داشت.

رویکرد دوم آن است که کتابخانه در سطح هسته پیاده‌سازی شود که به طور مستقیم توسط سیستم‌عامل پشتیبانی می‌شود. در این حالت، کد و ساختمان‌داده‌های کتابخانه در فضای هسته قرار دارند. احضار تابعی از API کتابخانه، یک فراخوانی سیستمی به هسته در پی خواهد داشت.

امروزه از سه کتابخانه‌ی نخ مهم استفاده می‌شود که عبارتند از: نخ‌های پوزیکس، ویندوز و جاوا. نخ‌های پوزیکس که توسعه‌ی نخ‌های پوزیکس استاندارد محسوب می‌شوند، ممکن است به صورت یک کتابخانه‌ی سطح کاربر و یا سطح هسته فراهم شوند. کتابخانه‌ی نخ ویندوز، یک کتابخانه‌ی سطح هسته است که در سیستم‌های ویندوز وجود دارد. ایجاد و مدیریت نخ در برنامه‌های جاوا به طور مستقیم توسط API نخ جاوا امکان‌پذیر است. با این وجود، چون در بیشتر موارد، JVM بالای یک سیستم‌عامل میزبان اجرا می‌شود، API نخ جاوا معمولاً با استفاده از یک کتابخانه‌ی نخ بر روی سیستم میزبان پیاده‌سازی می‌شود. این بدان معنی است که نخ‌های جاوا در سیستم‌های ویندوز معمولاً با استفاده از API ویندوز پیاده‌سازی می‌شوند؛ سیستم‌های یونیکس، لینوکس و macOS، نوعاً از نخ‌های پوزیکس استفاده می‌کنند.

در نخ‌سازی پوزیکس و ویندوز، هرگونه داده‌ای که به صورت سراسری اعلان شود–یعنی، در خارج هر یک از توابع اعلان شود–مابین تمامی نخ‌های متعلق به یک پردازه مشترک است. چون جاوا هیچ مفهوم معادلی از داده‌های سراسری ندارد، بایستی دسترسی داده‌های مشترک به صورت صریح میان نخ‌ها ترتیب یابد.

در باقی این بخش، اساس ایجاد نخ را با استفاده از این سه کتابخانه توصیف می‌کنیم. به عنوان یک مثال روشن‌کننده، یک برنامه‌ی چندنخی طراحی می‌کنیم که مجموع اعداد صحیح غیرمنفی را در یک نخ مجزا با استفاده از تابع حاصل‌جمع آشنای زیر انجام می‌دهد:

$$sum = \sum_{i=1}^{N} i$$

برای مثال، اگر N برابر ۵ باشد، این تابع مجموع اعداد ۱ تا ۵ را نمایش می‌دهد که برابر ۱۵ خواهد بود. هر یک از سه برنامه با ورودی کران بالای مجموع که از خط فرمان وارد می‌شود، اجرا می‌شوند. بنابراین، اگر کاربر عدد ۸ را وارد کند، در خروجی، مجموع مقادیر صحیح ۱ تا ۸ به نمایش در خواهد آمد.

پیش از این که به مثال‌های ایجاد نخ بپردازیم، دو راهبرد کلی ایجاد نخ‌های متعدد را معرفی می‌کنیم: نخ‌سازی ناهمگام و نخ‌سازی همگام. در نخ‌سازی ناهمگام، همین که والد نخ فرزندی ایجاد می‌کند، والد اجرای خود را از سر می‌گیرد، لذا والد و فرزند اجرای همروند و مستقل از یکدیگر دارند. چون نخ‌ها مستقل هستند، نوعاً اشتراک داده‌ی اندکی میان آن‌ها وجود دارد. نخ‌سازی ناهمگام راهبردی است که در کارگزار چندنخی نشان داده شده در شکل ۴-۲ استفاده شده و به‌صورت رایج نیز برای طراحی رابط‌های کاربری پاسخگو استفاده می‌شود.

نخ‌سازی همگام زمانی اتفاق می‌افتد که نخ والد، یک یا چندین فرزند ایجاد کرده و در ادامه بایستی قبل از این که کار خود را از سر بگیرد، منتظر ختم تمامی فرزندان خود بماند. در اینجا، نخ‌های ایجاد شده توسط والد، کار خود را به صورت همروند انجام می‌دهند، اما مادامی که کار آنها تمام نشده است، والد نمی‌تواند کار خود را از سر بگیرد. همین که هر یک از نخ‌ها کارشان را تمام کردند، خاتمه پذیرفته و به والد خود ملحق می‌شوند. تنها بعد از این که تمامی فرزندان بتوانند ملحق شدند، والد اجرا را از سر می‌گیرد. نوعاً، نخ‌سازی همگام مشمول اشتراک قابل توجه داده‌ها مابین نخ‌هاست. برای مثال، نخ والد ممکن است نتایج حاصل از فرزندان گوناگون خود را ترکیب کند. تمامی مثال‌های زیرین از نخ‌سازی همگام استفاده می‌کنند.

۴-۴-۱ نخ‌های پوزیکس

نخ‌های پوزیکس به استاندارد پوزیکس (IEEE 1003.1c) که برای ایجاد و همگامی نخ، یک API تعریف می‌کند اطلاق می‌شود. این یک *توصیف* برای رفتار نخ بوده و پیاده‌سازی نیست. طراحان سیستم‌عامل با هر روشی که بخواهند می‌توانند توصیف را پیاده‌سازی کنند. سیستم‌های متعددی که عمدتاً سیستم‌هایی از نوع یونیکس هستند از جمله لینوکس و macOS، توصیف نخ‌های پوزیکس را پیاده‌سازی می‌نمایند. هرچند که ویندوز به صورت بومی، نخ‌های پوزیکس را پشتیبانی نمی‌کند، یک سری پیاده‌سازی شخص ثالث برای ویندوز وجود دارد.

برنامه‌ی C نشان داده شده در شکل ۴-۱۱، API اساسی نخ‌های پوزیکس را جهت ساخت یک برنامه‌ی چندنخی که مجموع اعداد صحیح غیرمنفی را در یک نخ مجزا محاسبه می‌کند، نشان می‌دهد. در هر برنامه از نخ‌های پوزیکس، نخ‌های مجزا اجرای خود را در تابع مشخصی شروع می‌کنند. این تابع در شکل ۴-۱۱، runner() است. زمانی‌که این برنامه شروع می‌شود، یک نخ کنترل منفرد در main() آغاز می‌شود. بعد از یک سری مقداردهی اولیه، تابع main()، نخ دومی ایجاد می‌کند که کنترل را در تابع runner() به‌دست می‌آورد. هر دو نخ از sum به عنوان داده‌ی سراسری مشترک استفاده می‌کنند.

بیایید این برنامه را از نزدیک تحلیل کنیم. تمامی برنامه‌هایی که از نخ‌های پوزیکس استفاده می‌کنند بایستی فایل سرآیند pthread.h را بیاورند. دستور pthread_t tid شناسه‌ی نخی را که ایجاد خواهیم کرد تعریف می‌کند. هر نخ مجموعه صفاتی چون اندازه‌ی پشته و اطلاعات زمانبندی دارد. اعلان pthread_attr_t attr صفات نخ را تعریف می‌کند. صفات نخ را در فراخوانی تابع pthread_attr_init(&attr) انتخاب می‌کنیم. چون هیچ صفتی را آشکارا انتخاب نکرده‌ایم، از صفات پیش‌فرض موجود استفاده می‌کنیم (در فصل ۵، برخی صفات زمانبندی را که توسط API نخ‌های

پوزیکس فراهم می‌شود بحث می‌کنیم). با استفاده از فراخوانی تابع pthread_create() نخ مجزایی ایجاد می‌شود. علاوه بر شناسه و صفات نخ، نام تابعی را هم که نخ جدید، اجرایش را در آن آغاز خواهد نمود—در اینجا، تابع runner()—ارسال می‌کنیم. در نهایت، پارامتر صحیحی را که از طریق خط فرمان وارد می‌شود، یعنی argv[1] ارسال می‌کنیم.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */
int main(int argc, char *argv[]){
        pthread_t tid; /* the thread identifier */
        pthread_attr_t attr; /* set of thread attributes */

        /* set the default attributes of the thread */
        pthread_attr_init(&attr);
        /* create the thread */
        pthread_create(&tid,&attr,runner, argv[1]);
        /* wait for the thread to exit */
        pthread_join(tid,NULL);

        printf("sum = %d\n",sum);
}
/* The thread will execute in this function */
void *runner(void *param)
{
        int i, upper = atoi(param);
        sum = 0;
        for (i = 1; i <= upper; i++)
                sum += i;

        pthread_exit(0);
}
```

شکل ۴-۱۱ برنامه‌ی چندنخی C با استفاده از API نخ‌های پوزیکس.

از اینجا به بعد، برنامه دو نخ دارد که عبارتند از: نخ آغازین (یا والد) در main() و نخی که عملیات مجموع را در تابع runner() انجام می‌دهد (یا فرزند). این برنامه از راهبرد ایجاد نخ/join پیروی می‌کند که در آن، نخ والد بعد از ایجاد نخ جمع‌کننده، با فراخوانی تابع pthread_join() به انتظار تکمیل آن خواهد ماند. نخ جمع‌کننده با فراخوانی تابع pthread_exit() خاتمه خواهد یافت. همین که نخ جمع‌کننده کارش تمام شد، نخ والد مقدار داده‌ی مشترک sum را نمایش خواهد داد.

این برنامه‌ی نمونه فقط یک نخ منفرد ایجاد می‌کند. با رشد فراگیر سیستم‌های چندهسته‌ای، نوشتن برنامه‌هایی که نخ‌های متعددی را شامل هستند به صورت روزافزون رواج می‌یابد. یک روش ساده برای انتظارکشی نخ‌های متعدد، استفاده از تابع pthread_join() در میان یک حلقه‌ی for ساده است. برای مثال، می‌توانید با استفاده از کد نخ پوزیکس نشان داده شده در شکل ۴-۱۲ به ده نخ ملحق شوید.

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(workers[i], NULL);
```

شکل ۴-۱۲ کد نخ پوزیکس برای ملحق‌شدن به ده نخ.

۴-۴-۲ نخ‌های ویندوز

تکنیک ایجاد نخ‌ها با استفاده از کتابخانه‌ی نخ ویندوز از جهات متعددی مشابه تکنیک نخ‌های پوزیکس می‌باشد. API نخ ویندوز را با استفاده از یک برنامه‌ی C که در شکل ۴-۱۳ نشان داده می‌شود، روشن می‌کنیم. توجه داشته باشید هر زمان که از API ویندوز استفاده می‌کنیم بایستی فایل سرآیند windows.h را include کنیم.

درست مثل نسخه‌ی نخ‌های پوزیکس که در شکل ۴-۱۱ نشان داده شد، داده‌های مشترک نخ‌های مجزا—در اینجا، Sum—به صورت سراسری تعریف می‌شوند (نوع داده‌ی DWORD، یک عدد صحیح ۳۲ بیتی بدون علامت می‌باشد). تابع Summation() را هم تعریف می‌کنیم که در نخ مجزایی انجام می‌شود. به این تابع یک اشاره‌گر void ارسال می‌شود که در ویندوز به صورت LPVOID تعریف می‌شود. نخ مجری این تابع، مجموع مقادیر ۰ تا پارامتر ارسالی به تابع Summation() را در داده‌ی سراسری Sum قرار می‌دهد.

نخ‌های API ویندوز با استفاده از تابع CreateThread() ایجاد می‌شوند و—درست مثل نخ‌های پوزیکس—مجموعه‌ای از صفات نخ نیز به این تابع ارسال می‌شود. این صفات شامل اطلاعات امنیتی، اندازه‌ی پشته و پرچمی است که اگر قرار باشد نخ با حالت تعلیق کارش را شروع کند انتخاب می‌شود. در این برنامه، از مقادیر پیش‌فرض این صفات استفاده می‌کنیم. (مقادیر پیش‌فرض، در ابتدا نخ را به حالت تعلیق نبرده و برای اجرا توسط زمانبند پردازنده در حالت آماده قرار می‌دهند.) نخ والد با ایجاد نخ جمع‌کننده می‌بایست قبل از نمایش خروجی Sum، منتظر تکمیل آن بماند، چرا که مقدار آن توسط نخ جمع‌کننده به‌دست می‌آید. به یاد دارید که نخ والد در برنامه‌ی نخ پوزیکس (شکل ۴-۱۱) با استفاده از دستور pthread_join() منتظر نخ جمع‌کننده می‌ماند. معادل این کار را در API ویندوز با استفاده از تابع WaitForSingleObject() انجام می‌دهیم که باعث می‌شود نخ ایجادکننده تا خروج نخ جمع‌کننده مسدود شود.

در شرایطی که نیاز به انتظارکشی تکمیل نخ‌های متعدد است از تابع WaitForMultipleObjects() استفاده می‌شود. به این تابع، چهار پارامتر ارسال می‌شوند که عبارتند از:

۱- تعداد اشیائی که قرار است انتظارکشی شوند.
۲- اشاره‌گری به آرایه‌ی اشیاء
۳- پرچمی که نشان می‌دهد آیا تمامی اشیاء سیگنال داده می‌شوند یا خیر.
۴- یک فاصله‌ی timeout (یا INFINITE)

```
#include <windows.h>
#include <stdio.h>
DWORD Sum; /* data is shared by the thread(s) */
/* the thread will execute in this function */
DWORD WINAPI Summation(LPVOID Param) {
        DWORD Upper = *(DWORD*)Param;
        for (DWORD i = 0; i <= Upper; i++)
                Sum += i;
        return 0;
}
int main(int argc, char * argv[]) {
        DWORD ThreadId;
        HANDLE ThreadHandle;
        int Param;

        Param = atoi(argv[1]);
        /* create the thread */
        ThreadHandle = CreateThread(
                NULL, /* default security attributes */
                0, /* default stack size */
                Summation, /* thread function */
                &Param, /* parameter to thread function */
                0, /* default creation flags */
                &ThreadId); /* returns the thread identifier */

        /* now wait for the thread to finish */

        WaitForSingleObject(ThreadHandle, INFINITE);

        /* close the thread handle */
        CloseHandle(ThreadHandle);

        printf("sum = %d\n",Sum);
}
```

شکل ۴-۱۳ برنامه‌ی چندنخی C با استفاده از API ویندوز.

برای مثال، اگر THandles، آرایه‌ای از اشیاء Handle نخ به اندازه‌ی N باشد، نخ والد می‌تواند با دستور زیر به انتظار تکمیل تمامی نخ‌های فرزند خود بماند:

```
WaitForMultipleObjects(N, THandles, TRUE, INFINITE);
```

۴-۴-۳ نخ‌های جاوا

نخ‌ها مدل اساسی اجرای برنامه در یک برنامه‌ی جاوا هستند و زبان جاوا و API آن جهت ایجاد و مدیریت نخ‌ها مجموعه‌ای غنی از امکانات را فراهم می‌کنند. تمامی برنامه‌های جاوا حداقل از یک نخ کنترل منفرد تشکیل می‌شوند—حتی یک برنامه‌ی ساده‌ی جاوا که تنها یک متد main() دارد به صورت یک نخ منفرد در JVM اجرا می‌شود. نخ‌های جاوا در هر سیستمی که یک JVM دارد از جمله ویندوز، لینوکس و macOS در دسترس است. API نخ جاوا برای برنامه‌های اندروید نیز فراهم است.

برای ایجاد نخ‌ها در برنامه‌های جاوا دو تکنیک وجود دارند. رویکرد اول، ایجاد کلاس جدیدی است که از کلاس Thread مشتق شده و متد run() آن را بازنویسی می‌کند[1]. یک تکنیک دیگر—که به صورت رایج هم استفاده دارد—تعریف کلاسی است که رابط Runnable را پیاده‌سازی می‌کند. این رابط یک متد انتزاعی با امضای public void run() تعریف می‌کند. کد متد run() یک کلاس که Runnable را پیاده‌سازی می‌کند چیزی است که در یک نخ جداگانه‌ای اجرا می‌شود. یک مثال در زیر نشان داده می‌شود:

```
class Task implements Runnable
{
        public void run() {
                System.out.println("I am a thread.");
        }
}
```

عبارت‌های LAMBDA در جاوا

زبان جاوا با شروع از نسخه‌ی ۱/۸ عبارت‌های Lambda را معرفی نمود که یک نحو تمیزتری برای ایجاد نخ‌های فراهم می‌کند. به‌جای تعریف یک کلاس جداگانه‌ای که Runnable را پیاده‌سازی می‌کند، عبارت Lambda می‌تواند در عوض آن استفاده شود:

```
Runnable task = () -> {
        System.out.println("I am a thread.");
};

Thread worker = new Thread(task);
worker.start();
```

عبارت‌های Lambda—همین‌طور توابع مشابه تحت عنوان **بستار**[2]—یک ویژگی مهم زبان‌های

1. Override
2. Closures

برنامه‌نویسی تابعی است و در چندین زبان غیرتابعی نیز هم می‌باشد اعم از پایتون، C++، و C#. همان‌گونه که در مثال‌های بعدی در این فصل خواهیم دید، عبارت‌های Lambda اغلب یک نحو ساده‌ای برای نوشتن برنامه‌های موازی فراهم می‌کنند.

ایجاد نخ در جاوا مشمول ایجاد یک شئ Thread و ارسال یک نمونه از یک کلاس به آن است که Runnable را پیاده‌سازی می‌کند و به‌دنبال آن احضار متد ()start بر روی شئ Thread.

```
Thread worker = new Thread(new Task());
worker.start();
```

احضار متد ()start برای شئ جدید Thread دو کار انجام می‌دهد:

۱- حافظه تخصیص داده و یک نخ جدیدی را در JVM مقداردهی می‌نماید.
۲- متد ()run را فراخوانی می‌کند که با این کار نخ را برای اجرا توسط JVM آماده می‌نماید. (باز هم توجه دارید که ما هرگز متد ()run را به‌صورت مستقیم فراخوانی نمی‌کنیم. بلکه، متد ()start را فراخوانی می‌کنیم که آن متد ()run را از طرف ما فراخوانی می‌کند.)

به یاد بیاورید که نخ‌های والد در کتابخانه‌های پوزیکس و ویندوز ()pthread_join و ()WaitForSingleObject (به‌ترتیب) برای انتظارکشی اتمام نخ‌های جمع‌کننده قبل از پیشروی استفاده می‌کردند. متد ()join در جاوا کارکرد مشابهی را فراهم می‌کند. (توجه داشته باشید که ()join می‌تواند استثنای InterruptedException را پرتاب کند که ما از آن صرف‌نظر می‌کنیم.)

```
try {
        worker.join();
}
catch (InterruptedException ie) { }
```

اگر قرار باشد که والد منتظر اتمام چندین نخ باشد، متد ()join می‌تواند در یک حلقه‌ی for شبیه به آنچه که در نخ‌های پوزیکس در شکل ۴-۱۲ نشان داده شد، قرار بگیرد.

۴-۴-۳-۱ چارچوب Executor جاوا

جاوا از زمان شروعش، ایجاد نخ را با استفاده از رویکردی که تشریح کردیم پشتیبانی کرده است. با این وجود، با شروع از نسخه‌ی ۱/۵ و API آن، جاوا چندین امکان همروندی جدیدی معرفی کرد که کنترل بسیار بیشتری در ایجاد و ارتباط نخ در اختیار برنامه‌نویسان می‌گذارند. این ابزار در بسته‌ی java.util.concurrent در دسترس قرار می‌گیرند.

به‌جای ایجاد صریح اشیاء Thread، ایجاد نخ در عوض، حول رابط Executor سازمان‌دهی می‌شود:

```
public interface Executor
{
        void execute(Runnable command);
}
```

کلاس‌هایی که این رابط را پیاده‌سازی می‌کنند بایستی متد ()execute را تعریف کنند که شئ Runnable به آن ارسال می‌شود. این موضوع برای برنامه‌نویسان جاوا بدان معنی است که

به‌جای ایجاد یک شئ Thread جداگانه و احضار متد ()start آن، از Executor استفاده کنند. Executor به‌صورت زیر مورد استفاده قرار می‌گیرد:

```
Executor service = new Executor;
service.execute(new Task());
```

چارچوب Executor مبتنی بر مدل تولیدکننده–مصرف‌کننده می‌باشد؛ وظایفی که رابط Runnable را پیاده‌سازی می‌کنند تولید می‌شوند، و نخ‌هایی که این وظایف را اجرا می‌کنند آن‌ها را مصرف می‌کنند. حسن این رویکرد در آن است که نه تنها ایجاد نخ را از اجرا تفکیک می‌کند بلکه راهکاری برای ارتباط میان وظایف همروند فراهم می‌کند.

اشتراک داده‌ها میان نخ‌های متعلق به پردازه‌ای یکسان در ویندوز و پوزیکس به‌راحتی اتفاق می‌افتد، چرا که داده‌های مشترک به‌سادگی به‌صورت سراسری اعلان می‌شوند. جاوا به‌عنوان یک زبان شئ‌گرای محض، هیچ مفهومی از داده‌های سراسری این شکلی ندارد. ما می‌توانیم پارامترهایی را به کلاسی که Runnable را پیاده‌سازی می‌کند ارسال کنیم، اما نخ‌های جاوا نمی‌توانند نتایج را برگردانند. برای رفع این نیاز، بسته‌ی java.util.concurrent رابط اضافی Callable را تعریف می‌کند که شبیه به Runnable رفتار می‌کند جز این که یک نتیجه هم می‌تواند برگردانده شود. نتایج برگردانده شده از وظایف Callable به اشیاء Future معروف هستند. یک نتیجه می‌تواند از متد ()get تعریف شده در رابط Future بازیابی شود. برنامه‌ی نشان داده شده در شکل ۴–۱۴ برنامه‌ی مجموع را با استفاده از این امکانات جاوا نشان می‌دهد.

```
import java.util.concurrent.*;
class Summation implements Callable<Integer>
{
        private int upper;
        public Summation(int upper) {this.upper = upper;}

        /* The thread will execute in this method */
        public Integer call() {
                int sum = 0;
                for (int i = 1; i <= upper; i++) sum += i;
                return new Integer(sum);
        }
}

public class Driver
{
        public static void main(String[] args) {
        int upper = Integer.parseInt(args[0]);
        ExecutorService pool = Executors.newSingleThreadExecutor();
        Future<Integer> result = pool.submit(new Summation(upper));

        try {
```

```
            System.out.println("sum = " + result.get());
        } catch (InterruptedException | ExecutionException ie) {}
        }
}
```

شکل ۴-۱۴ نمایش API چارچوب Executor جاوا.

کلاس Summation رابط Callable را پیاده‌سازی می‌کند که متد ()call را توصیف می‌کند—کد این متد ()call است که در یک نخ جداگانه‌ای اجرا می‌شود. ما برای اجرای این کد، یک شئ newSingleThreadExecutor (که به‌صورت یک متد ایستا در کلاس Executors فراهم می‌شود) ایجاد می‌کنیم که از نوع ExecutorService می‌باشد و یک وظیفه‌ی Callable با استفاده از متد ()submit آن به آن ارسال می‌کنیم. (تفاوت اصلی میان متدهای ()execute و ()submit در این است که اولی هیچ نتیجه‌ای بر نمی‌گرداند، در حالی که دومی، نتیجه‌ای از نوع یک Future بر می‌گرداند.) به‌محض این که وظیفه‌ی callable را به نخ ارسال می‌کنیم، با فراخوانی متد ()get شئ Future که بر می‌گرداند منتظر نتیجه‌اش می‌مانیم.

کاملاً ساده است که در نگاه اول متوجه شد که این مدل ایجاد نخ، به‌ظاهر پیچیده‌تر از ایجاد ساده‌ی یک نخ و الحاق به آن با خاتمه‌اش می‌باشد. با این وجود، تحمل این عوارض نسبتاً کم در مقابل منافع آن ارزشمند است. همان‌گونه که دیدیم، استفاده از Callable و Future به نخ‌ها امکان می‌دهد تا نتایجی را برگردانند.

به‌علاوه، این رویکرد، ایجاد نخ‌ها را از نتایجی که تولید می‌کنند تفکیک می‌کند: والد به‌جای این که منتظر ختم یک نخ بماند قبل از این که نتایج را بازیابی کند، در عوض، فقط منتظر می‌ماند تا نتایج در دسترس قرار بگیرند. در نهایت، همان‌گونه که در بخش ۴-۵-۱ خواهیم دید، این چارچوب می‌تواند با سایر امکانات ترکیب شود تا ابزار قدرتمندی برای مدیریت تعداد بسیار زیادی نخ فراهم شود.

JVM و سیستم‌عامل میزبان

JVM نوعاً در بالای سیستم‌عامل میزبان پیاده‌سازی می‌شود (شکل ۱۸-۱۰ را ببینید). این برپایی باعث می‌شود تا JVM جزئیات پیاده‌سازی سیستم‌عامل زیرین را مخفی کرده و یک محیط سازگار و انتزاعی فراهم نماید تا برنامه‌های جاوا بتوانند در هر بستری که از JVM پشتیبانی می‌کند عملیات داشته باشند. توصیف JVM، چگونگی نگاشت نخ‌های جاوا را به سیستم‌عامل زیرین نشان نمی‌دهد، بلکه این تصمیم‌گیری را به پیاده‌سازی خاص JVM می‌سپارد. برای مثال، سیستم‌عامل ویندوز از مدل یک-به-یک استفاده می‌کند؛ بنابراین، هر نخ جاوا برای یک JVM که بر روی ویندوز اجرا می‌شود به نخی از هسته نگاشته می‌شود. به علاوه، ممکن است یک رابطه هم میان کتابخانه‌ی نخ جاوا و کتابخانه‌ی نخ سیستم‌عامل میزبان به‌وجود بیاید. برای مثال، پیاده‌سازی‌های یک JVM برای خانواده‌ی سیستم‌عامل‌های ویندوز امکان دارد که موقع ایجاد نخ‌های جاوا از API ویندوز استفاده کنند؛ سیستم‌های لینوکس و macOS ممکن است از API نخ‌های پوزیکس بهره ببرند.